

BATTLE SIMULATOR

Documentação Técnica e Manual de Uso

- **Instituição:** Universidade Federal de Sergipe (UFS)
- **Disciplina:** Algoritmo e Estrutura de Dados I
- **Componentes (Grupo 14):**
 - Thierry Nathan Chagas Silva Costa
 - Cauã Souza Corumba
 - João Vitor Souza

1. OBJETIVOS

O *Battle Simulator* é uma aplicação desktop estruturada sob o paradigma de Orientação a Objetos que simula dinamicamente combates em turnos, inspirados em mecânicas clássicas de jogos de RPG (*Role-Playing Games*) da era 8-bit. O ecossistema computacional coloca um protagonista controlado pelo usuário contra hordas procedurais e escaláveis compostas por 2 a 5 inimigos simultâneos por rodada.

O objetivo principal e central deste projeto consiste em substituir integralmente o ecossistema de coleções nativas e utilitárias da API do Java (tais como **ArrayList** ou **LinkedList**) por uma estrutura de dados proprietária de baixo nível. Para isso, foi implementada uma Fila Circular Encadeada Dinâmica auto-referenciável, encarregada da gerência dos combatentes ativos e da ordenação estrita do ciclo de turnos (*game loop*).

A escolha da linguagem Java introduz um fator de design defensivo essencial ao projeto: embora a plataforma abstraia o gerenciamento manual de memória física (através do **Garbage Collector**) e proíba a aritmética direta de ponteiros brutos, o cerne do trabalho reside na manipulação abstrata de referências em tempo de execução. A conexão, inserção e remoção dos nós de combate ocorrem por meio do acoplamento lógico de links auto-referenciados, simulando com rigor o comportamento de ponteiros estruturais de memória.

Dessa forma, o sistema busca demonstrar a viabilidade de construir uma aplicação gráfica robusta, reativa e aderente ao padrão arquitetural MVC Estrito, sustentada exclusivamente por uma estrutura de dados customizada e isolada de facilitadores nativos da linguagem.

2. FERRAMENTAS

A aplicação foi construída utilizando o ecossistema tecnológico Java moderno,

dividindo-se entre lógica computacional pura e uma camada robusta de interface gráfica de usuário (GUI):

- **Linguagem:** Java 21, fazendo uso intensivo de recursos recentes da linguagem, tais como *Java Records* para transporte imutável de dados.
- **Interface Gráfica:** JavaFX 21, permitindo a renderização assíncrona, desacoplada e reativa das janelas do simulador.
- **Estruturação Visual:** FXML (XML dialetal do JavaFX) para a árvore de componentes visuais (**battle-view.fxml**).
- **Estilização Temática:** CSS3 customizado (**battle.css**) aplicando uma identidade visual limpa com tipografia monoespçada integrada para emular o estilo *retro-gaming*.

3. DETALHES DA IMPLEMENTAÇÃO E ENGENHARIA DE SOFTWARE

3.1 Arquitetura de Software (MVC Estrito)

O projeto adota o padrão de arquitetura Model-View-Controller (MVC) estrito. Isso significa que o motor lógico do jogo não possui qualquer conhecimento sobre botões, telas ou barras de vida. Toda a comunicação ocorre por meio de padrões de transferência de dados imutáveis (*Payloads*), garantindo robustez de *runtime* e impedindo que a interface altere por acidente o estado interno das entidades.

3.2 A Estrutura de Dados Central: CircularQueue

A mecânica de turnos, o gerenciamento do jogador vivo e a horda ativa de inimigos são governados por uma Fila Circular Simplesmente Encadeada com Nó Cabeça. O nó cabeça mitiga erros clássicos de manipulação de ponteiros ao eliminar a necessidade de tratar ponteiros nulos em filas vazias.

A rotação do loop de combate não exige reinicialização de arrays ou busca linear. Graças à natureza circular da fila, o método **rotateTurn()** simplesmente remove o atacante atual da cabeça da fila e o insere novamente no fim, gastando tempo constante **O(1)**.

3.3 Abordagem Defensiva e Evolução do Projeto

Como medida de segurança de *runtime*, todos os métodos de mutação estrutural aplicam validações sob o princípio *Fail-Fast*. Parâmetros nulos injetados na fila disparam rejeição instantânea antes da alteração dos ponteiros de memória.

Durante a evolução do projeto, as listas originais de personagens baseadas em **ArrayList** foram completamente retrabalhadas e substituídas pela **CircularQueue** customizada. Essa transição eliminou bugs recorrentes de indexação e unificou a engenharia de dados do software. Além disso, o fluxo de reinício e transição de rounds foi reestruturado para que, ao derrotar o último monstro, os controles visuais limpem os

inimigos remanescentes e gerem novos botões dinamicamente, proporcionais ao tamanho da nova horda.

4. EVIDÊNCIAS DE CÓDIGO E DESTAQUES TÉCNICOS

Abaixo estão discriminados os trechos de códigos vitais desenvolvidos pela equipe, servindo de material analítico para o crescimento e estudo compartilhado da turma.

4.1 Métodos de Instanciação Dinâmica e Balanceamento

EnemyFactory.generateHorde(int currentRound)

O método **generateHorde** gerencia o desafio do cenário através da geração procedural de ameaças, determinando tanto a quantidade de oponentes quanto o nível de poder individual de cada criatura.

```
public Enemy[] generateHorde(int currentRound) {  
    int enemiesQty = (int) (Math.random() * (maxEnemies - minEnemies + 1) + minEnemies);  
  
    int multiplier = currentRound-1;  
    int currentMinMaxHealth = baseMinHealth + (multiplier * healthGrowth);  
    int currentmaxMaxHealth = baseMaxHealth + (multiplier * healthGrowth);  
    int currentminAttackDamage = baseMinAttack + (multiplier * attackGrowth);  
    int currentmaxAttackDamage = baseMaxAttack + (multiplier * attackGrowth);  
  
    Enemy[] enemiesList = new Enemy[enemiesQty];  
    for (int i = 0; i < enemiesQty; i++) {  
        int maxHealth = (int) (Math.random() * (currentmaxMaxHealth - currentMinMaxHealth + 1) + currentMinMaxHealth);  
        int attackDamage = (int) (Math.random() * (currentmaxAttackDamage - currentminAttackDamage + 1) + currentminAttackDamage);  
  
        Enemy enemy = new Enemy( name: "Inimigo" + (i + 1), maxHealth, attackDamage);  
        enemiesList[i] = enemy;  
    }  
    return enemiesList;  
}
```

PlayerFactory.generatePlayer(int currentRound)

O método **generatePlayer** é responsável por instanciar a unidade que representa o herói controlado pelo usuário a cada nova etapa de batalha.

```

public Player[] generatePlayer(int currentRound){ 1 usage  🧑 Cauã Corumba
    int playersQuantity = 1;

    int multiplier = currentRound-1;
    int currentMaxHealth = baseHealth + (multiplier * healthGrowth);
    int currentAttackDamage = baseAttack + (multiplier * attackGrowth);

    Player[] players = new Player[playersQuantity];
    for(int i = 0; i < playersQuantity; i++){

        int maxHealth = currentMaxHealth;
        int attackDamage = currentAttackDamage;

        Player player = new Player( name: "Hero" + (i+1), maxHealth, attackDamage);
        players[i] = player;
    }
    return players;
}

```

4.2 Métodos de Processamento de Turnos Autônomos e Atualização Visual Dinâmica

BattleEngine.executeEnemyAttack()

Quando o topo da fila encadeada (**CircularQueue**) aponta que o atacante ativo é uma entidade do tipo Combatant, o sistema abdica da interação do usuário e invoca internamente o processamento autônomo.

```

private void executeEnemyAttack(){ 1 usage 8 Cauã Corumba +2
    BattleStatusRecord status = battleEngine.executeEnemyTurn();
    String killedName = null;

    battleLogView.getItems().add(status.actionLog());

    if (status.isGameOver() && status.killedTarget() != null) {
        killedName = status.killedTarget().name();
        battleLogView.getItems().add("Derrota... " + killedName + " tombou em combate");
        updateAllUI();

        controlsContainer.getChildren().clear();
        Button restartButton = new Button(s: "Jogar Novamente");
        restartButton.getStyleClass().add("action-button");
        restartButton.setOnAction( ActionEvent e -> handleGameRestart());
        controlsContainer.getChildren().add(restartButton);
    }
    else {
        updateAllUI();
        resetControls();
    }
    battleLogView.scrollTo(i: battleLogView.getItems().size() - 1);
}

```

BattleEngine.executePlayerAttack(CombatantRecord target)

Ao contrário do fluxo automatizado dos oponentes, a ofensiva do protagonista é desencadeada por um evento direto da interface gráfica, disparado quando o utilizador seleciona um alvo válido no menu dinâmico de botões.

```

private void executePlayerAttack(CombatantRecord target) { 1 usage  ⤴ Cauã Corumba +1
    BattleStatusRecord status = battleEngine.executeAttack(target.id());

    battleLogView.getItems().add(status.actionLog());

    if (status.isGameOver() && status.isVictory()) {
        battleLogView.getItems().add("Vitória! A horda foi derrotada! Avançando de round...");
        startNextRound();
        updateAllUI();
        resetControls();
    }
    else {
        updateAllUI();
        resetControls();
    }
    battleLogView.scrollTo(battleLogView.getItems().size() - 1);
}

```

BattleController.updateEnemySprites()

A cada modificação de estado lida pelo controlador, a interface gráfica precisa ser redesenhada para mitigar inconsistências visuais (como exibir a imagem de um monstro que já foi derrotado). O método **updateEnemySprites** resolve o ciclo de vida visual das hordas na tela.

```

private void updateEnemySprites() { 1 usage  ⤴ João Vitor
    enemySpritesContainer.getChildren().clear();

    List<CombatantRecord> enemies = battleEngine.getEnemiesRecords();

    for (CombatantRecord enemy : enemies) {
        if (!enemy.isDead()) {
            javafx.scene.layout.Region sprite = new javafx.scene.layout.Region();
            sprite.getStyleClass().add("enemy-sprite");
            enemySpritesContainer.getChildren().add(sprite);
        }
    }
}

```

4.3 Métodos de Métodos de Sincronização e Renderização Reativa de Componentes Dinâmicos

BattleController.updateTurnOrder()

O método **updateTurnOrder** é responsável por atualizar o carrossel visual que exibe a ordem dos próximos turnos na interface gráfica (**turnOrderContainer**).

```
public void updateTurnOrder() { 1 usage  João Vitor +1
    turnOrderContainer.getChildren().clear();
    List<CombatantRecord> turnOrder = battleEngine.getTurnOrderRecords();
    List<Label> carouselLabels = TurnOrderFactory.createCarouselNodes(turnOrder);
    turnOrderContainer.getChildren().addAll(carouselLabels);
}
```

updateEnemyLifeBars

Ao contrário do jogador, a horda de ameaças possui comportamento multifacetado e dinâmico, exigindo um ciclo de varredura iterativo sobre o container horizontal oposto (**enemyStatusContainer**).

updatePlayerLifeBar

A sincronização dos atributos vitais do protagonista é direcionada especificamente para o painel de controle do usuário (**playerStatusContainer**).

```
private void updateEnemyLifeBars() { 1 usage  Thierry Costa +2
    enemyStatusContainer.getChildren().clear();
    List<CombatantRecord> enemies = battleEngine.getEnemiesRecords();
    for (CombatantRecord enemy : enemies) {
        VBox enemyBox = CombatantStatusFactory.createStatusNode(enemy, Pos.TOP_LEFT, textFirst: false);
        enemyStatusContainer.getChildren().add(enemyBox);
    }
}

private void updatePlayerLifeBar() { 1 usage  Thierry Costa +2
    playerStatusContainer.getChildren().clear();
    List<CombatantRecord> players = battleEngine.getPlayersRecords();
    for (CombatantRecord player : players) {
        VBox playerBox = CombatantStatusFactory.createStatusNode(player, Pos.BOTTOM_RIGHT, textFirst: true);
        playerStatusContainer.getChildren().add(playerBox);
    }
}
```

5. MANUAL DE OPERAÇÃO DA APLICAÇÃO

Para executar e operar o simulador de batalhas corretamente, siga o fluxo de instruções operacionais detalhado a seguir:

■ Etapa 0: Clonagem do repositório

- **Ação:** clone o repositório para o seu desktop
- **Comando:** `"git clone https://github.com/thierry-costa-ufs/battle-simulator"`

■ Etapa 1: Inicialização

- **Ação:** Execute a classe **BattleApplication.java** na sua IDE ou terminal.
- **Comportamento:** A janela principal (800x600px) abre carregando o *Carrossel de Turnos* no topo da aplicação, o *Herói* embaixo à esquerda e a *Horda de Goblins* em cima à direita.

■ Etapa 2: Turno do Jogador

- **Ação:** Identifique o seu turno no carrossel. O painel inferior direito exibirá botões dinâmicos para atacar e, depois, selecionar os alvos do ataque com os nomes dos alvos da horda atual.
- **Comportamento:** Os botões de seleção de alvo redimensionam-se automaticamente ocupando todo o espaço disponível do painel de controle. Clique no botão do inimigo que deseja golpear.

■ Etapa 3: Processamento e Log

- **Ação:** Após clicar no alvo, acompanhe as atualizações instantâneas no *painel de log*.
- **Comportamento:** O dano é calculado, a barra de vida do inimigo reduz proporcionalmente e o log de batalha adiciona uma nova linha detalhada descrevendo a ação do herói.

■ Etapa 4: Turno do Inimigo

- **Ação:** Quando o turno passar para um monstro, o botão principal muda de forma e exibe o texto "Próximo Turno". Clique nele.
- **Comportamento:** O sistema do jogo realiza automaticamente o ataque do oponente escolhendo o jogador ativo, calcula o contra-ataque baseado nos atributos do round atual e atualiza a barra de vida do Herói, registrando a ação no log.

■ Etapa 5: Progressão de Round (Level Up)

- **Ação:** Derrote o último inimigo vivo da horda atual para finalizar o combate em andamento.
- **Comportamento:** O sistema limpa os painéis de controle, restaura os pontos de vida do Herói, incrementa os seus atributos base e gera uma nova horda de goblins mais poderosa para o round seguinte.

■ Etapa 6: Fim de Jogo e Reinício

- **Ação:** Ocorre caso os pontos de vida do Herói cheguem a zero.
- **Comportamento:** O painel de log exibe que atingiu o estado de derrota com a opção de reiniciar a partida a partir do Round 1.

Nota de Configuração de Ambiente: Certifique-se de que os arquivos de recursos estáticos (**battle-view.fxml** e **battle.css**) estão devidamente associados à pasta de *resources* da sua estrutura de compilação na IDE para assegurar que os estilos visuais e as imagens funcionem perfeitamente sem lançar exceções de Input/Output.